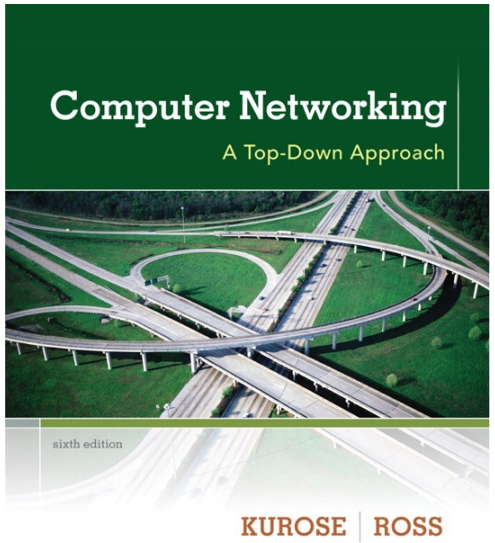
Goal: The following code implements a client-server applications using UDP. Analyse the source code and run the application. Insert comments in code.




# *Computer Networking: A Top Down Approach*

6th edition
Jim Kurose, Keith Ross
Addison-Wesley
March 2012

## A note on the use of these ppt slides:

We're making these slides freely available to all (faculty, students, readers). They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a *lot* of work on our part. In return for use, we only ask the following:

- If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

Thanks and enjoy! JFK/KWR

©All material copyright 1996-2012
J.F Kurose and K.W. Ross, All Rights Reserved

# Socket programming *with UDP*

UDP: no “connection” between client and server

- no handshaking
- sender explicitly attaches IP address and port of destination to each packet
- server must extract IP address, port of sender from received packet

UDP: transmitted data may be received out of order, or lost

application viewpoint

UDP provides unreliable transfer of groups of bytes (“datagrams”) between client and server

# Example: UDP client

from socket import *

*Socket module forms the basis of all network communications in Python. By including this line, we will be able to create sockets within our program.*

serverName = 'hostname'

*String serverName set to hostname. Either the IP address of the server (e.g., "128.138.32.126") or the host-name of the server (e.g., "cis.poly.edu") should be provided. If we use the hostname, then a DNS lookup will automatically be performed to get the IP address.)*

serverPort = 12000

*Sets the integer variable serverPort to 12000.*

clientSocket = socket(AF_INET, SOCK_DGRAM)

*Creates the client's socket called clientSocket. First parameter is the address family (AF_INET for IPv4). Second parameter is the socket type (SOCK_DGRAM for UDP socket). The port number of the client socket is given by the operating system.*

message = raw_input('Input lowercase sentence:')

*raw_input() is a built-in function in Python. The user is prompted with the words "Input data:"*

clientSocket.sendto(message,(serverName, serverPort))

*Method sendto() attaches the destination address (serverName, serverPort) to the message and sends the packet into the process's socket, clientSocket. The source address is also attached to the packet, although this is done automatically rather than explicitly by the code.*

modifiedMessage, serverAddress = clientSocket.recvfrom(2048)

*When a packet arrives from the Internet at the client's socket, the packet's data is put into the variable modifiedMessage and the packet's source address is put into the variable serverAddress. The variable serverAddress contains both the server's IP address and the server's port number.*

print modifiedMessage

*Prints out modifiedMessage on the user's display.*

clientSocket.close()

# Example: UDP server

from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)

*Beginning of UDPServer is similar to UDPClient. It also imports the socket module, also sets the integer variable serverPort to 12000, and also creates a socket of type SOCK_DGRAM (a UDP socket).*

serverSocket.bind((*'', serverPort))*

*Assigns the port number 12000 to the server's socket. The code is explicitly assigning a port number to the socket. When anyone sends a packet to port 12000 at the IP address of the server, that packet is directed to this socket. The '' allows the socket to be associated with a specific client IP.*

print *"The server is ready to receive"*
while 1:

*While loop will allow UDPServer to receive and process packets from clients indefinitely*

message, clientAddress = serverSocket.recvfrom(2048)

*When a packet arrives at the server's socket, the packet's data is put into the variable message and the packet's source address is put into the variable clientAddress. The variable clientAddress contains both the client's IP address and the client's port number. UDPServer will make use of this address information, as it provides a return address.*

modifiedMessage = message.upper()

*Capitalizes the message received.*

serverSocket.sendto(modifiedMessage, clientAddress)

*Attaches the client's address (IP address and port number) to the capitalized message, and sends the resulting packet into the server's socket (the server address is also attached to the packet, although this is done automatically rather than explicitly by the code.)*